

**МИНИСТЕРСТВО НАУКИ, ВЫСШЕГО ОБРАЗОВАНИЯ
И ИННОВАЦИЙ КЫРГЫЗСКОЙ РЕСПУБЛИКИ**
**КЫРГЫЗСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ им. И. РАЗЗАКОВА**
ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
Кафедра «Программное обеспечение компьютерных систем»

ОТЧЁТ

ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

на тему:

**«ТЕСТИРОВАНИЕ МУЛЬТИМЕДИЙНЫХ И РЕЛИГИОЗНЫХ МОДУЛЕЙ
ОБРАЗОВАТЕЛЬНОГО ПРИЛОЖЕНИЯ IRFAN ACADEMY»**

Выполнил: Салибаев Нурислам
Группа: ПИ(англ)-2-23
Проверила: Валеева А. А.

КАЛЕНДАРНЫЙ ГРАФИК ПРОХОЖДЕНИЯ ПРАКТИКИ

Таблица 1 — Календарный график и краткая характеристика выполненных работ

№	Срок выполнения	Содержание и краткая характеристика выполненных работ
1	16.06.26– 22.06.26	Изучение проекта и подготовка тестирования. Проанализирована структура Flutter-приложения, определены тестируемые модули, зависимости, риски и критерии успешного результата.
2	23.06.26– 29.06.26	Функциональное тестирование видеомодуля. Проверены выбор и отмена файла, автоматическая загрузка, прогресс, потоковый и блочный режимы, метаданные, Firestore и сетевой видеоплеер.
3	30.06.26– 06.07.26	Тестирование религиозных функций. Проверены геолокация и резервные координаты, расписание намазов, переход после Иша, направление Кыблы, компас и локальные уведомления.
4	07.07.26– 13.07.26	Платформенное и регрессионное тестирование. Выполнены проверки Android и Web, воспроизведены и повторно проверены дефекты, запущены flutter analyze, flutter test и node --check. Оформлены результаты.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 АНАЛИЗ ОБЪЕКТА И СРЕДСТВ ТЕСТИРОВАНИЯ.....	6
1.1 Цель и задачи тестирования.....	6
1.2 Характеристика тестируемых модулей	7
1.3 Границы ответственности участника.....	7
1.4 Требования и критерии качества.....	8
1.5 Среда и инструменты тестирования	9
1.5.1 Flutter и Dart.....	10
1.5.2 Firebase	11
1.5.3 Clouinary.....	11
1.5.4 Firebase Functions	11
1.5.5 GitHub.....	12
1.5.6 Android и Web.....	12
1.6 Пакеты и зависимости тестируемых модулей	13
1.7 Структура тестируемых компонентов	13
2 ТЕСТИРОВАНИЕ МОДУЛЯ ЗАГРУЗКИ И ВОСПРОИЗВЕДЕНИЯ	
ВИДЕО	15
2.1 Объект, условия и критерии тестирования видеомодуля	15
2.2 Проверка выбора файла и автоматического запуска загрузки.	16
2.3 Проверка потоковой и блочной загрузки файла.....	17
2.4 Проверка метаданных и сохранения записи	18
2.5 Проверка сетевого видеоплеера	19
2.6 Негативные сценарии и обработка ошибок	21
3 ТЕСТИРОВАНИЕ ГЕОЛОКАЦИИ, РАСПИСАНИЯ НАМАЗОВ,	
КЫБЛЫ И УВЕДОМЛЕНИЙ	22
3.1 Объект и набор проверок религиозного модуля.....	22
3.2 Проверка получения географических координат	24
3.3 Проверка расписания и следующего намаза.....	25
3.4 Проверка направления Кыблы.....	26

3.5	Проверка локальных напоминаний	26
3.6	Платформенное тестирование на Android и Web	27
4	АВТОМАТИЗИРОВАННОЕ И РЕГРЕССИОННОЕ ТЕСТИРОВАНИЕ	29
4.1	Методика и уровни тестирования	29
4.2	Среда и результаты автоматизированных проверок	29
4.3	Автоматизированное тестовое покрытие	30
4.4	Регрессионная проверка видеомодуля.....	31
4.5	Регрессионная проверка религиозного модуля	31
4.6	Регистрация и повторная проверка дефектов	32
5	ИТОГОВЫЕ РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ.....	33
5.1	Результаты тестирования видеомодуля	33
5.2	Результаты тестирования религиозного модуля.....	33
5.3	Сводный результат тестирования	33
	ЗАКЛЮЧЕНИЕ	35
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
	ПРИЛОЖЕНИЕ А.....	38
	А.1 Файлы, исследованные при тестировании	38
	ПРИЛОЖЕНИЕ Б. ЧЕК-ЛИСТ ДЕМОНСТРАЦИИ	39

ВВЕДЕНИЕ

Во время производственной практики мной выполнялось тестирование мультимедийных и религиозных функций приложения Irfan Academy. Объектами проверки стали загрузка и воспроизведение видеоуроков, получение геолокации, расчёт времени намазов, направление Кыблы и локальные уведомления. Основное внимание уделялось функциональным, негативным и платформенным сценариям.

Цель работы — оценить корректность, устойчивость и удобство указанных модулей на Android и Web. Для каждого модуля определялись входные условия, действия пользователя, ожидаемый результат и фактическая реакция приложения. Обнаруженные дефекты воспроизводились повторно после исправления.

Проверка сочетала ручное функциональное тестирование, анализ граничных состояний, статический анализ Flutter, автоматизированные тесты и проверку синтаксиса Firebase Functions. Результаты зафиксированы в таблицах и подтверждены снимками экранов приложения и терминала.

1 АНАЛИЗ ОБЪЕКТА И СРЕДСТВ ТЕСТИРОВАНИЯ

1.1 Цель и задачи тестирования

Цель индивидуальной работы заключалась в комплексной проверке мультимедийных и религиозных модулей Irfan Academy. Требовалось подтвердить корректную работу основных пользовательских сценариев, устойчивость при сетевых и платформенных ограничениях, а также отсутствие регрессии после устранения обнаруженных ошибок.

Для достижения цели были выполнены следующие задачи:

- изучить архитектуру приложения и определить границы тестируемых модулей;
- составить набор позитивных, негативных и граничных сценариев;
- проверить выбор видеофайла, отмену выбора и автоматический запуск загрузки;
- проверить индикатор прогресса, сообщения об ошибках и повторные попытки;
- проверить потоковую и блочную передачу видео разных размеров;
- сверить URL, длительность, размер, формат и другие метаданные с результатом загрузки;
- проверить запуск, паузу, перемотку, повтор и изменение скорости видеоплеера;
- проверить геолокацию при разрешённом и запрещённом доступе, тайм-ауте и недоступной службе;
- проверить расписание намазов и переход к Фаджру следующего дня после Иша;
- проверить направление Кыблы при наличии и отсутствии датчика компаса;
- проверить локальные уведомления на Android и безопасную работу Web-версии;

- выполнить flutter analyze, flutter test, node --check и повторную регрессионную проверку.

1.2 Характеристика тестируемых модулей

Irfan Academy представляет собой образовательное приложение на Flutter, использующее единую кодовую базу для мобильной и Web-версии. Приложение объединяет учебные материалы и религиозные функции. В рамках индивидуального задания рассматривались только компоненты, связанные с видеоуроками, геолокацией, временем намаза, направлением Кыблы и локальными уведомлениями.

Мультимедийный модуль предназначен для выбора видео на устройстве, его отправки во внешнее хранилище, получения технических метаданных и последующего воспроизведения по защищённому HTTPS-адресу. В базу данных при этом должны передаваться только сведения о видео, а не сам бинарный файл. Такой подход предотвращает чрезмерное потребление памяти приложения и не создаёт избыточную нагрузку на Firestore.

Религиозный модуль предназначен для получения координат пользователя, формирования расписания намазов с ханафитским временем Асра, выбора следующего намаза и вычисления направления на Каабу. Локальные напоминания дополняют расписание, но включаются только там, где соответствующий платформенный API поддерживается.

1.3 Границы ответственности участника

Зоной ответственности третьего участника являлось тестирование мультимедийных и религиозных функций. Работа включала подготовку сценариев, выполнение проверок, фиксацию фактических результатов, воспроизведение дефектов и контроль исправлений для следующих компонентов:

- сервис загрузки видео во внешнее хранилище;
- сценарий выбора файла и отображения состояния загрузки;

- сетевой видеоплеер и обработку его состояний;
- сервис получения координат пользователя;
- расчёт расписания намазов и следующего намаза;
- виджет определения направления Кыблы;
- локальные напоминания о предстоящих намазах;
- проверку перечисленных функций в Android- и Web-версиях.

Сохранение учебных метаданных выполняется через существующий слой проекта. Изменение Firebase Authentication, проектирование коллекций Firestore, управление ролями и разработка остальных функций административной панели в индивидуальную задачу не включались.

1.4 Требования и критерии качества

Критерии качества были сформированы на основе реальных пользовательских действий. Проверялось, что приложение не требует ручного ввода технических данных видео, показывает состояние длительной операции, корректно реагирует на отказ разрешений и не вызывает неподдерживаемые платформенные API.

- выбор видеофайла непосредственно с устройства без ручного ввода ссылки;
- автоматический запуск загрузки после подтверждения выбора;
- отображение процента передачи, возможность отмены и повторной попытки;
- поддержка крупных файлов без чтения всего содержимого в оперативную память;
- автоматическое получение размера, формата, адреса и продолжительности видео;
- реальное воспроизведение с обработкой загрузки, паузы и ошибки ресурса.

К религиозному модулю предъявлялись следующие требования:

- запрашивать геолокацию только после проверки системного разрешения;
- не блокировать экран при отказе пользователя или недоступности датчика;
- использовать резервные координаты Бишкека при невозможности определить местоположение;
- корректно определять следующий намаз, включая переход к Фаджру следующего дня;
- рассчитывать Кыблу по тем же координатам, что и расписание;
- не вызывать неподдерживаемое планирование уведомлений в браузере.

К нефункциональным требованиям были отнесены устойчивость к нестабильному соединению, понятные сообщения об ошибках, освобождение ресурсов видеоплеера, отсутствие секретных ключей в пользовательском интерфейсе и сохранение работоспособности существующих Firebase-модулей.

1.5 Среда и инструменты тестирования

Технологический стек Irfan Academy включает клиентскую платформу, облачные сервисы, внешнее видеохранилище, серверный интеграционный слой, систему контроля версий и целевые платформы. Их обобщённое назначение приведено в таблице 2.

Таблица 2 — Назначение основных технологий Irfan Academy

Технология	Роль в проекте	Применение при тестировании
Flutter / Dart	клиентская логика и пользовательский интерфейс	запуск приложения, статический анализ и проверка пользовательских сценариев Android и Web

Технология	Роль в проекте	Применение при тестировании
Firebase	Authentication и Cloud Firestore	проверка входа, ролей, чтения и сохранения метаданных учебных материалов
Cloudinary	внешнее видеохранилище и доставка файлов	проверка загрузки видео, ответа API, URL, public_id, размера и длительности
Firebase Functions	серверный слой для защищённых интеграций	проверка синтаксиса серверного файла командой <code>node --check</code>
GitHub	распределённый контроль версий	сопоставление изменений и повторная проверка после исправления дефектов
Android / Web	целевые платформы демонстрационной версии	проверка платформенных различий геолокации, компаса и уведомлений

1.5.1 Flutter и Dart

Flutter является основой клиентской части Irfan Academy. Он формирует экраны, диалоговые окна, списки уроков, индикатор загрузки, элементы управления видеоплеером и интерфейс религиозных модулей. Виджеты Flutter позволяют использовать одну структуру интерфейса на Android и в браузере, сохраняя единое визуальное поведение приложения.

Язык Dart используется для моделей данных, сервисов и асинхронных операций. Конструкции Future и Stream обеспечивают ожидание сетевых ответов, наблюдение за изменениями Firestore, обновление прогресса загрузки и управление жизненным циклом видеоплеера. Для проекта это означает, что клиентская логика и пользовательский интерфейс разрабатываются в одной кодовой базе.

1.5.2 Firebase

Firebase используется как основная облачная платформа приложения. Firebase Authentication выполняет регистрацию, вход и выход пользователей по электронной почте и паролю. Полученный уникальный идентификатор пользователя связывает учётную запись с профилем и персональными данными в базе.

Cloud Firestore хранит профили, роли, учебные записи, задания, расписание, уведомления и метаданные видеоуроков. Сам видеофайл в Firestore не передаётся: в коллекции videoLessons сохраняются только название, номер урока, продолжительность, размер, тип содержимого, Cloudinary public_id и сетевой адрес. Поточковые запросы Firestore позволяют интерфейсу автоматически отображать изменения данных.

1.5.3 Cloudinary

Cloudinary выполняет хранение и доставку видеоуроков в текущей демонстрационной версии. После передачи файла сервис возвращает secure_url для воспроизведения, public_id для идентификации ресурса, размер, формат и продолжительность. Благодаря этому технические характеристики не вводятся администратором вручную.

Файл передаётся через Upload API. Небольшие и средние видео отправляются потоком, а крупные разбиваются на части. Полученный HTTPS-адрес используется пакетом video_player. В проекте применён unsigned upload preset без размещения Cloudinary API Secret в клиенте.

1.5.4 Firebase Functions

Firebase Functions предназначены для операций, которые нельзя безопасно выполнять во Flutter-приложении. В каталоге functions подготовлены вызываемые серверные функции для интеграции с объектным хранилищем: начало и завершение multipart-загрузки, подпись отдельных частей, получение временной ссылки, чтение технических сведений и

удаление объекта. Перед выполнением сервер проверяет Firebase Authentication, а для изменяющих операций дополнительно проверяется роль администратора в Firestore.

Секретные ключи в этой схеме определяются только на стороне Functions и не включаются в APK или Web-сборку. В демонстрационной версии видео загружается напрямую в Cloudinary, поэтому подготовленные функции R2 не участвуют в рабочем потоке загрузки.

1.5.5 GitHub

GitHub используется как удалённое хранилище исходного кода и основа командного контроля версий. Репозиторий фиксирует изменения в виде коммитов, позволяет сопоставлять выполненную работу с конкретными файлами и возвращаться к стабильной версии при обнаружении ошибки. Для команды это обеспечивает единую историю проекта и уменьшает риск потери или случайной перезаписи результатов других участников.

1.5.6 Android и Web

Android и Web являются целевыми платформами демонстрационной версии. На Android доступны системная геолокация, аппаратный компас и планирование локальных уведомлений. В Web-версии выбор файла и воспроизведение выполняются средствами браузера, а сетевые запросы дополнительно зависят от политики CORS.

Единая кодовая база не означает полного равенства платформенных API. Поэтому перед вызовом локальных уведомлений выполняется проверка kIsWeb, при отсутствии датчика компаса интерфейс показывает безопасное состояние, а при недоступной геолокации используются резервные координаты. Такой подход позволяет сохранить общий интерфейс, но учесть реальные ограничения каждой платформы.

1.6 Пакеты и зависимости тестируемых модулей

Кроме основных технологий, индивидуальная часть использует набор специализированных Flutter-пакетов. В таблице 3 перечислены только библиотеки, которые непосредственно участвуют в загрузке и воспроизведении видео либо в работе религиозных модулей.

Таблица 3 — Библиотеки, использованные в индивидуальной части проекта

Инструмент	Версия	Назначение
Flutter и Dart	Dart 3.11.5	разработка общей кодовой базы для Android и Web
Cloudinary Upload API	Web API	хранение и получение сетевого адреса видео
image_picker	1.2.3	выбор видеофайла на устройстве пользователя
http	1.6.0	потокковая и частичная отправка видеофайлов
video_player	2.14.0	воспроизведение видео по сетевому адресу
geolocator	14.0.3	получение координат и обработка разрешений
adhan	2.0.0+1	расчёт времени намаза и направления Кыблы
flutter_compass	0.8.1	получение направления устройства
flutter_local_notifications	22.0.1	локальные напоминания на поддерживаемых платформах

Выбранное сочетание библиотек разделяет ответственность между компонентами: сетевой сервис выполняет отставку видео, контроллер видеоплеера отвечает за воспроизведение, а геолокация и религиозные вычисления изолированы от пользовательского интерфейса. Это упрощает обработку ошибок и последующую проверку каждого модуля.

1.7 Структура тестируемых компонентов

Перед выполнением сценариев были изучены файлы `cloudinary_video_service.dart`, `prayer_service.dart`, `notification_service.dart`, `video_player_modal.dart` и `qibla_compass.dart`. Анализ позволил определить точки входа, зависимости, граничные условия и ожидаемую реакцию интерфейса.

Структура файлов индивидуальных модулей представлена на рисунке 1.

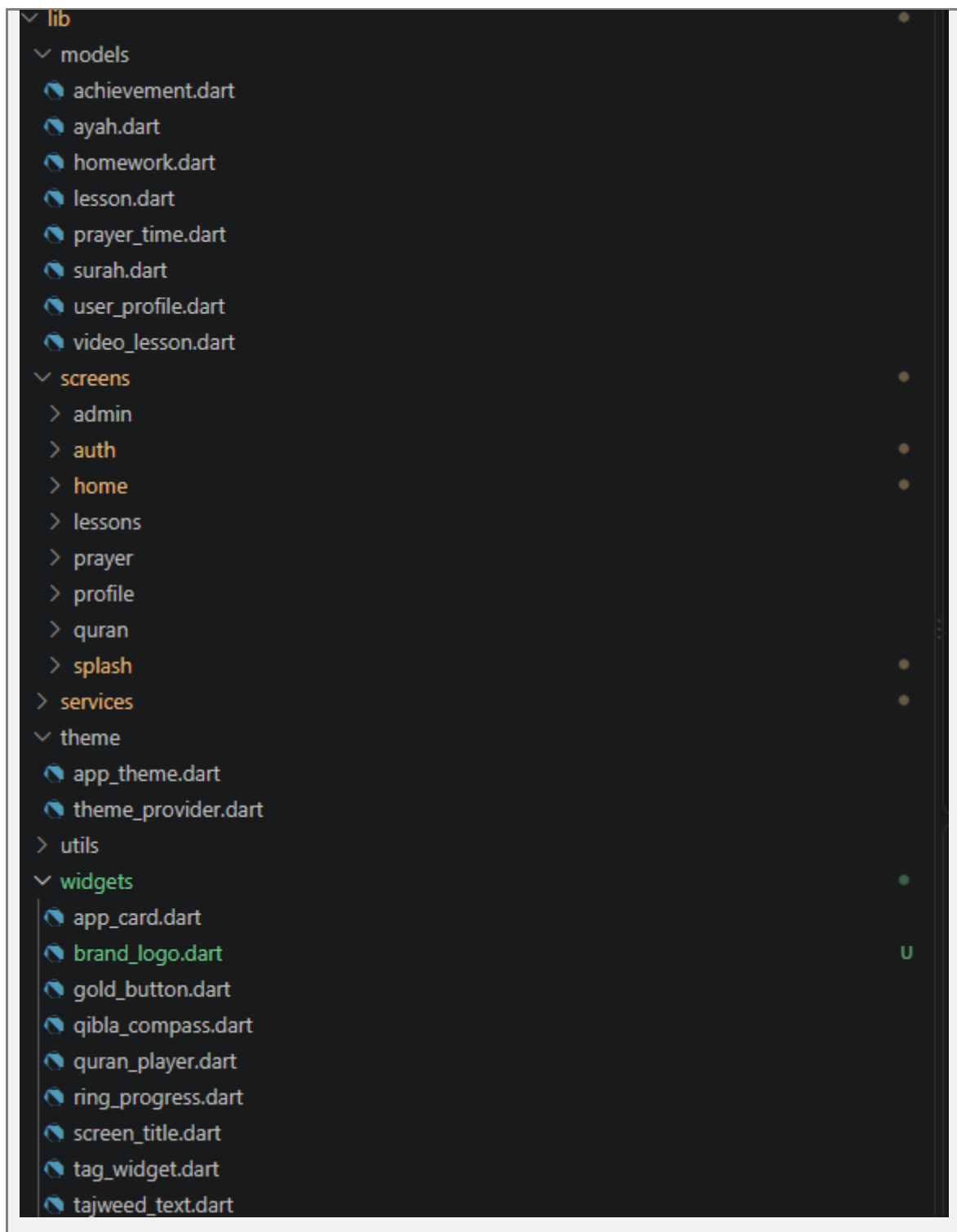


Рисунок 1 — Структура файлов мультимедийных и религиозных модулей

Irfan Academy

2 ТЕСТИРОВАНИЕ МОДУЛЯ ЗАГРУЗКИ И ВОСПРОИЗВЕДЕНИЯ ВИДЕО

2.1 Объект, условия и критерии тестирования видеомодуля

Объектом тестирования стал полный путь видеоурока: выбор файла администратором, передача в Cloudinary, получение технических данных, сохранение записи в Firestore и последующее воспроизведение. Для длинных уроков отдельно учитывались размер файла, длительность операции и потребление памяти.

Проверка выполнялась для фактически используемого пути Cloudinary. Подготовленные функции Cloudflare R2 не участвовали в рабочем сценарии, поэтому не включались в положительный результат тестирования. Ранее полученная ошибка CORS Firebase Storage учитывалась как исходный дефект Web-сценария.

Контрольные точки видеомодуля включали правильное разделение данных между Flutter, Cloudinary и Firestore, наличие обязательных полей и доступность полученного URL. Проверяемые компоненты приведены в таблице 4, а последовательность действий — на рисунке 2.

Таблица 4 — Проверяемые компоненты и данные видеомодуля

Компонент	Обрабатываемые данные	Назначение
Flutter-приложение	выбранный XFile, прогресс и временное состояние формы	управление выбором, отправкой и отображением результата
Cloudinary	бинарный видеофайл и его технические характеристики	постоянное хранение файла и доставка по HTTPS
Cloud Firestore	название урока, URL, public_id, размер, формат и длительность	хранение только метаданных для списка видеоуроков

Поток загрузки и воспроизведения видео показан на рисунке 2.

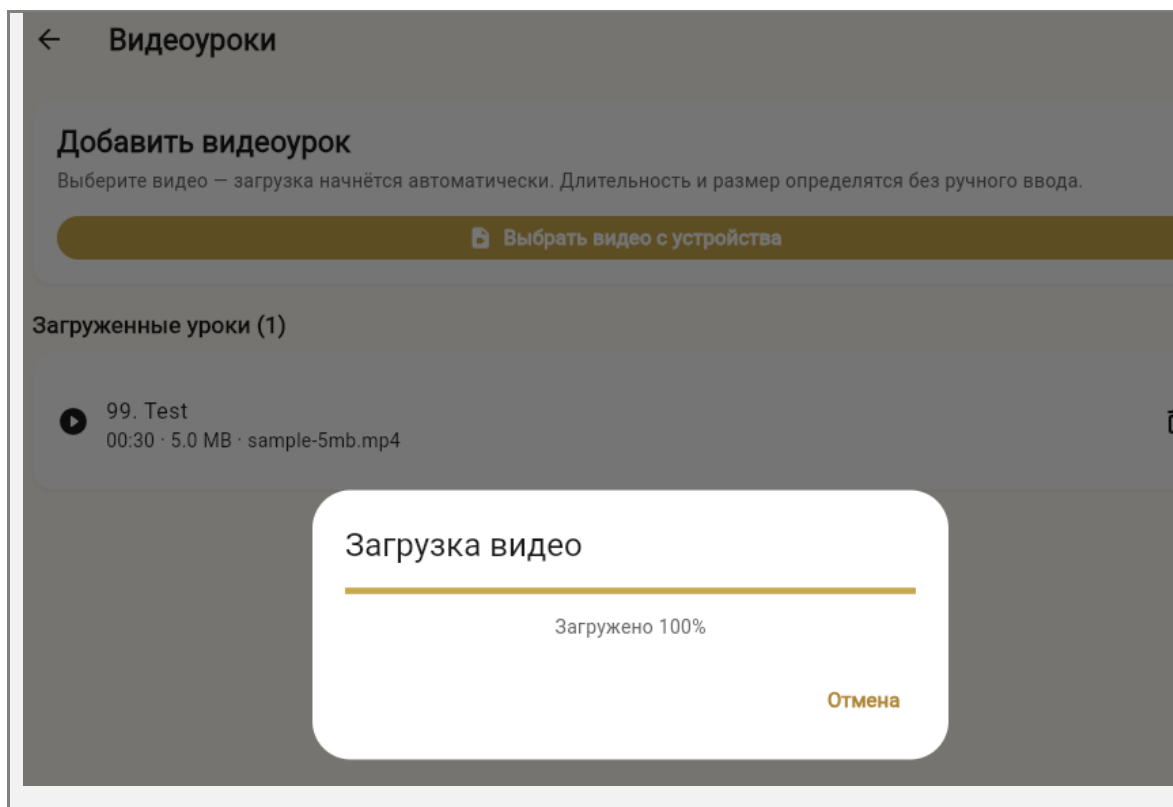


Рисунок 2 — Схема проверяемого пути видеоурока

2.2 Проверка выбора файла и автоматического запуска загрузки

Проверены два исхода системного выбора файла. При отмене форма сохраняла исходное состояние и не выводила ошибку. При выборе MP4 возвращался объект XFile, открывался защищённый диалог загрузки и начиналось обновление индикатора прогресса.

После успешной передачи проверялось отсутствие полей ручного ввода URL и длительности. Администратор заполнял только номер, название и описание урока, а размер, формат, длительность, `public_id` и адрес подставлялись из ответа Cloudinary. Фактическое поведение совпало с ожидаемым.

Экран выбора видео показан на рисунке 3.

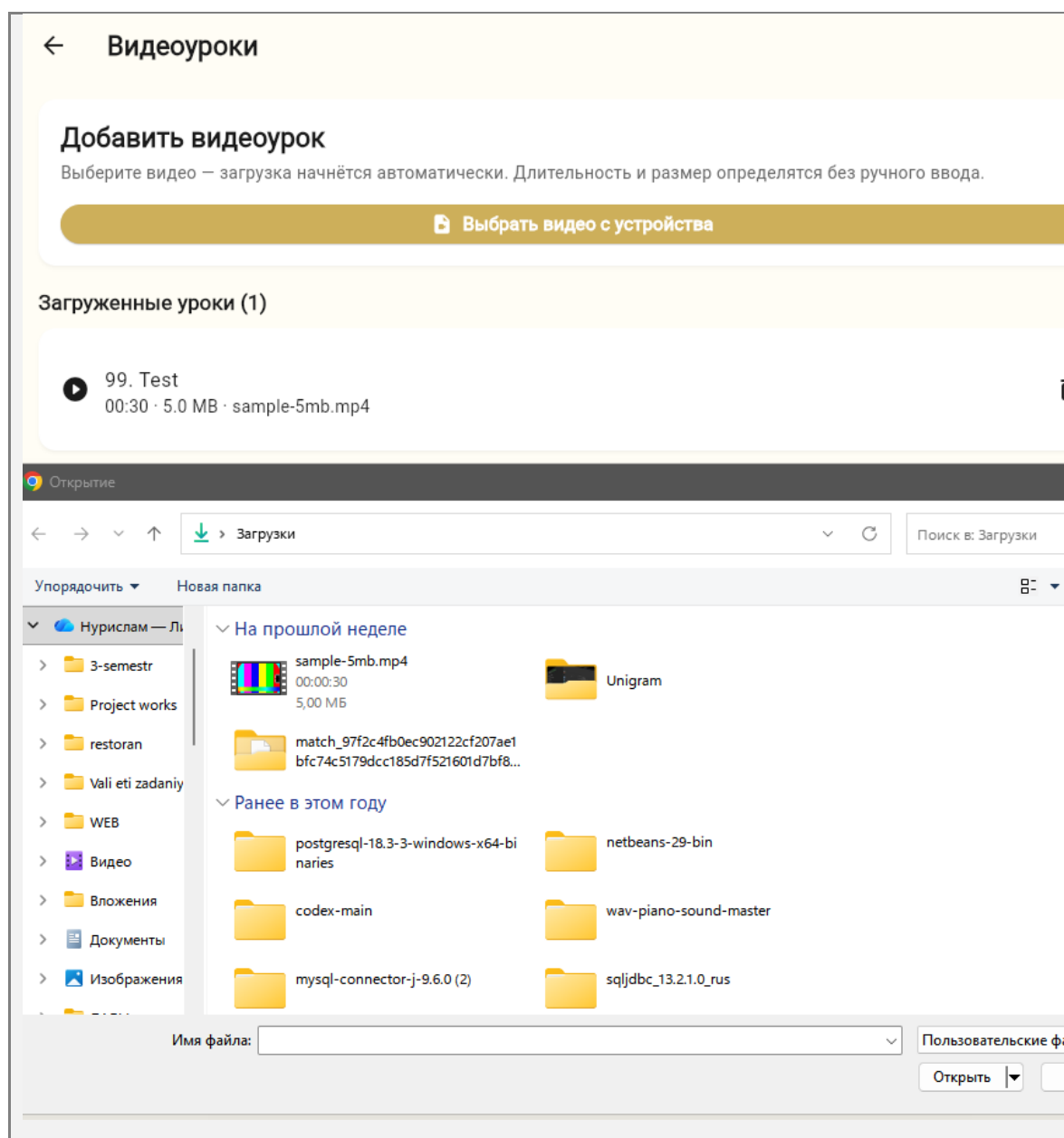


Рисунок 3 — Выбор видеофайла в административной панели

2.3 Проверка потоковой и блочной загрузки файла

Для файлов размером до 95 МБ проверялся потоковый `MultipartRequest`. Содержимое считывалось через `openRead`, индикатор изменялся по количеству отправленных байтов, а нулевой файл отклонялся до сетевого запроса. Загрузка всего файла в оперативную память не наблюдалась.

Для файла больше 95 МБ проверялся блочный режим. Последовательные диапазоны по 8 МБ передавались с общим `X-Unique-`

Upload-Id и корректным Content-Range. После последней части сервис возвращал единый результат, а общий прогресс достигал 100 %.

Сценарий временной сетевой ошибки подтвердил повтор части до четырёх попыток с задержками 1, 2 и 4 секунды. Ответы 4xx не повторялись автоматически, поскольку относились к ошибкам запроса. После успешной попытки индикатор продолжал увеличиваться без сброса.

Отображение прогресса загрузки показано на рисунке 4.

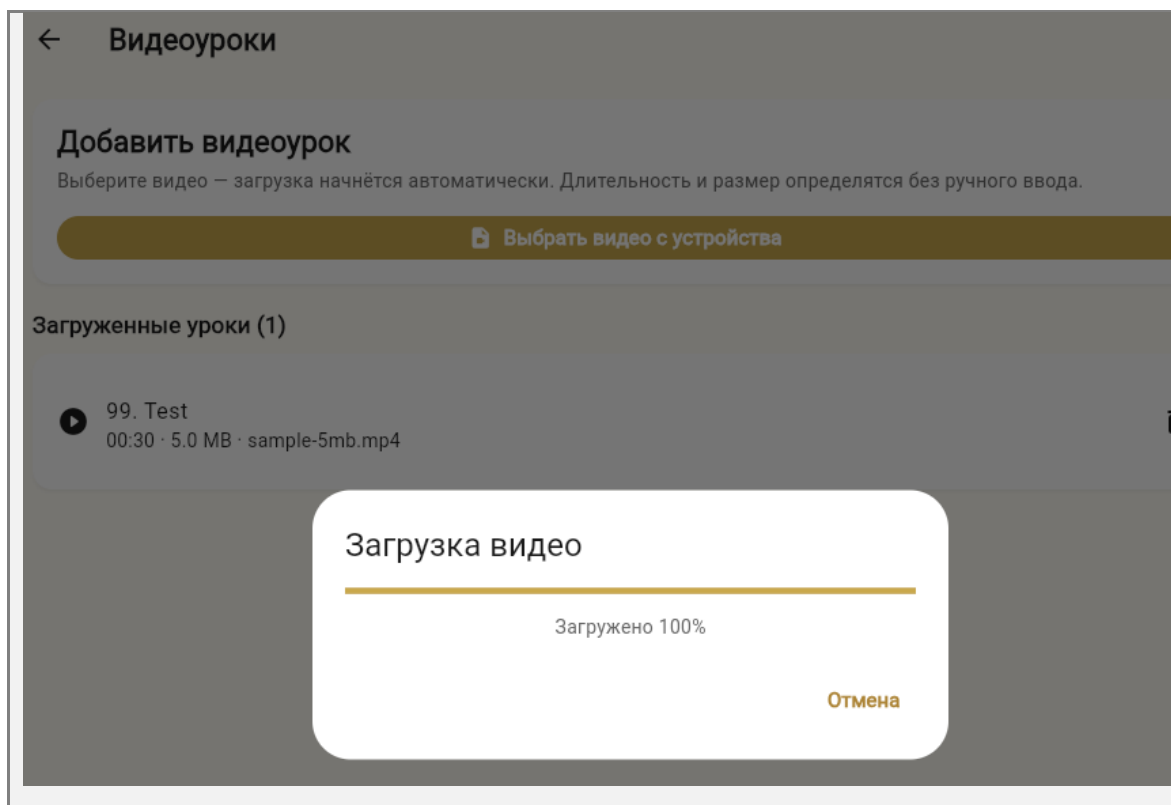


Рисунок 4 — Результат проверки прогресса загрузки большого видео

2.4 Проверка метаданных и сохранения записи

После завершения загрузки проверялся JSON-ответ Cloudinary. Успешным считался только результат с `secure_url` и `public_id`. Дополнительно сверялись `bytes`, `format` и `duration`, после чего значения преобразовывались в `CloudinaryUploadResult`. Перечень проверенных полей приведён в таблице 5.

Таблица 5 — Поля метаданных, проверенные после загрузки видео

Поле	Источник	Использование в приложении
<code>secureUrl</code>	<code>secure_url</code>	сетевой адрес для видеоплеера
<code>publicId</code>	<code>public_id</code>	идентификатор файла в Cloudinary
<code>fileName</code>	<code>XFile.name</code>	исходное имя выбранного файла

Поле	Источник	Использование в приложении
fileSize	bytes	размер файла в байтах
contentType	format	тип содержимого вида video/mp4
durationSeconds	duration	продолжительность, округлённая до секунд

В Firestore проверялся документ коллекции videoLessons. Он содержал учебные поля, метаданные Cloudinary, storageProvider со значением cloudinary и серверную дату создания. Бинарное содержимое видео в документе отсутствовало, что подтвердило правильное разделение хранилища и базы метаданных.

После создания документа проверялось обновление списка уроков и формирование уведомления о новом видеоуроке. Повторное открытие записи возвращало тот же адрес и технические данные. Результат зафиксирован на рисунке 5.

Результат сохранения видеоурока показан на рисунке 5.

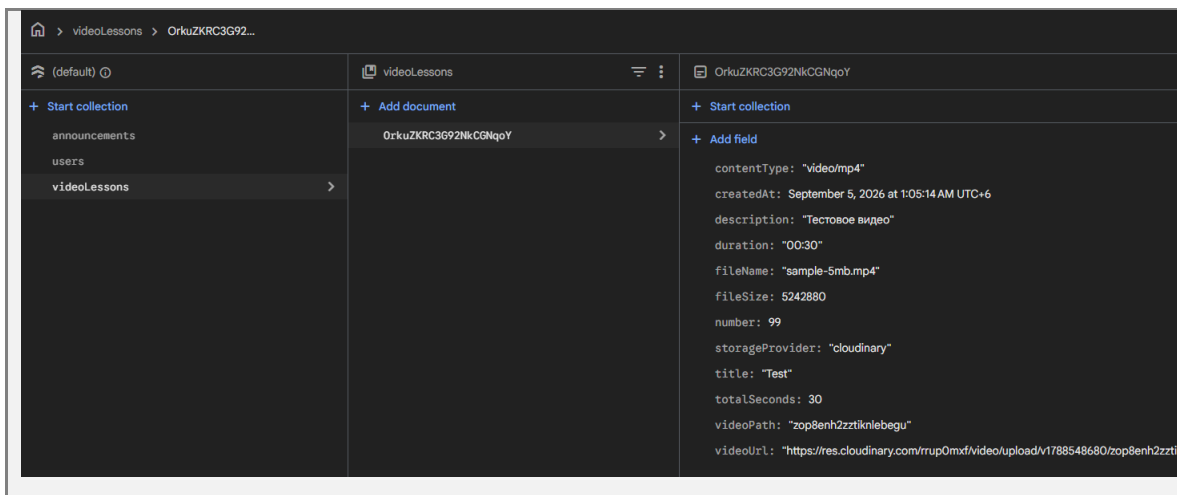


Рисунок 5 — Результат проверки загрузки и сохранения метаданных

2.5 Проверка сетевого видеоплеера

Проверка начиналась с открытия опубликованного урока по secure URL. Вместо прежней имитации таймером инициализировался VideoPlayerController.networkUrl, загружались реальные данные и отображалась фактическая продолжительность видеозаписи.

Последовательно проверялись запуск, пауза, перемотка ползунком, повтор, переход между уроками и изменение скорости. Для каждой команды

контролировались изображение, текущая позиция и состояние кнопок. Основные элементы управления работали ожидаемо.

Отдельно проверялись буферизация, недоступный URL, повторная инициализация и закрытие окна. При смене урока предыдущий listener и контроллер освобождались, поэтому поздний ответ старого запроса не изменял новый экран.

Работа сетевого видеоплеера показана на рисунке 6.

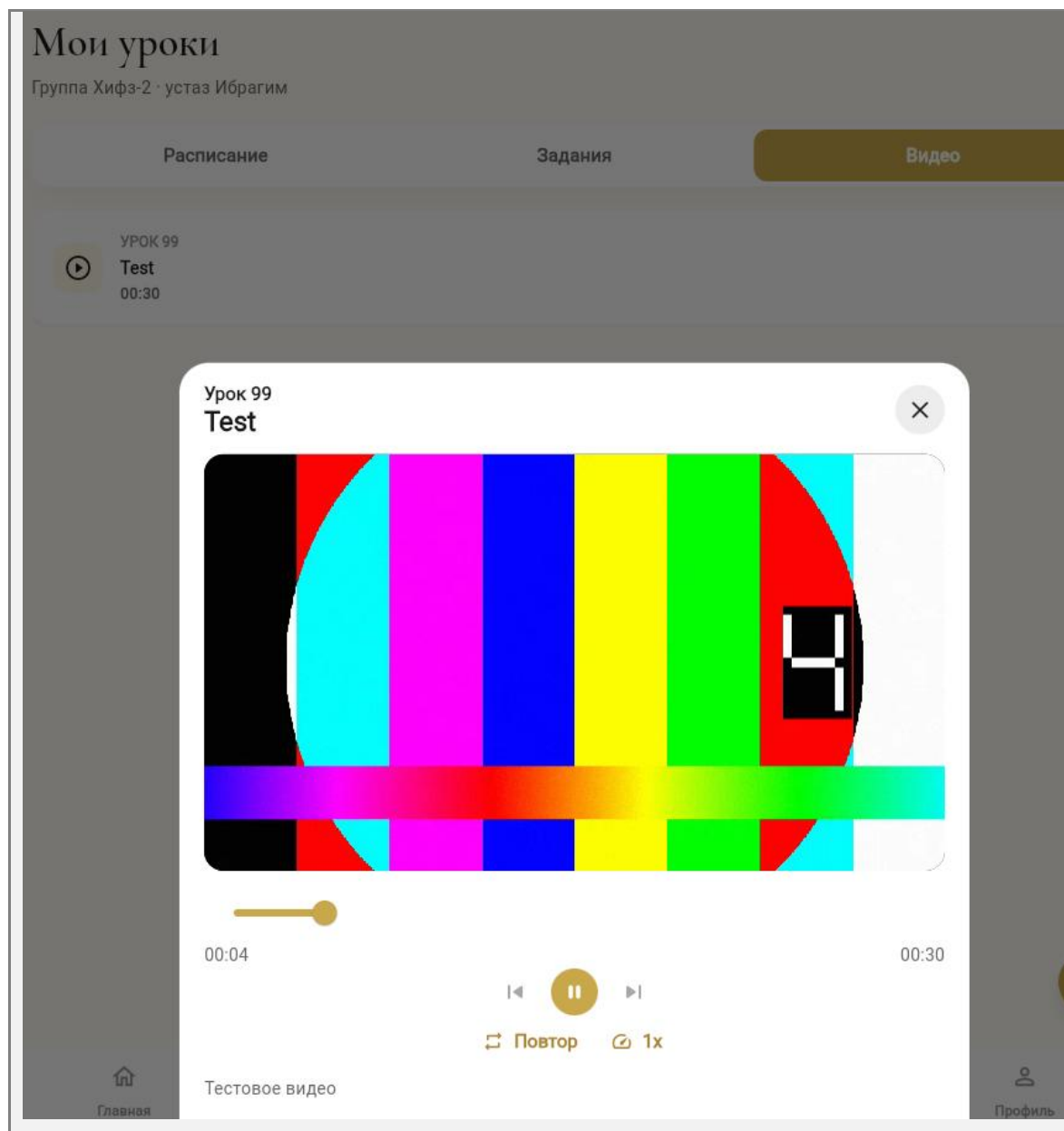


Рисунок 6 — Результат проверки сетевого видеоплеера

2.6 Негативные сценарии и обработка ошибок

Негативные сценарии охватывали пустой файл, отмену операции, ответ 4xx, временный сетевой сбой, некорректный ответ Cloudinary и недоступный URL плеера. Ожидаемая реакция и фактический результат систематизированы в таблице 6.

Таблица 6 — Негативные сценарии видеомодуля

Ситуация	Реакция программы	Результат для пользователя
выбран пустой файл	операция прекращается до сетевого запроса	отображается сообщение о некорректном файле
ответ Cloudinary 4xx	ошибка API разбирается и не повторяется автоматически	можно изменить файл или настройки и повторить загрузку
временный сбой части	выполняется до четырёх попыток с увеличением задержки	загрузка может продолжиться без повторного выбора файла
отмена пользователем	публикация урока не выполняется; цикл частей останавливается при проверке флага	диалог закрывается без создания записи видеоурока
некорректный или недоступный URL	контроллер освобождается и сохраняется состояние ошибки	показывается понятное сообщение и кнопка повторной попытки

Отмена проверялась до начала загрузки, между блоками и перед повторной попыткой. После установки флага новые сетевые действия не запускались, диалог закрывался управляемо, а незавершённый урок не добавлялся в список.

При анализе клиентской сборки подтверждено отсутствие Cloudinary API Secret. Используемый unsigned preset не содержал секретного ключа в APK или Web-коде. Эта проверка относилась к контролю конфигурации тестируемого демонстрационного сценария.

3 ТЕСТИРОВАНИЕ ГЕОЛОКАЦИИ, РАСПИСАНИЯ НАМАЗОВ, КЫБЛЫ И УВЕДОМЛЕНИЙ

3.1 Объект и набор проверок религиозного модуля

Объектами проверки религиозного модуля стали PrayerService, PrayerScreen, QiblaCompass и NotificationService. Сценарии охватывали положительные и отрицательные состояния геолокации, смену даты, переход после Иша, наличие и отсутствие компаса, а также различия Android и Web.

Таблица 7 — Проверяемые компоненты религиозного модуля

Компонент	Основная функция	Результат
PrayerService	геолокация, вычисление намазов и Кыблы	объект PrayerSchedule для выбранных координат
PrayerSchedule	объединение координат, направления и списка времён	единая модель данных экрана
PrayerTime	хранение названия, времени и состояния намаза	строка расписания с визуальным статусом
PrayerScreen	загрузка и периодическое обновление расписания	пользовательский экран намазов
QiblaCompass	сопоставление Кыблы и курса устройства	графический указатель направления
NotificationService	планирование будущих напоминаний	системные уведомления на поддерживаемой платформе

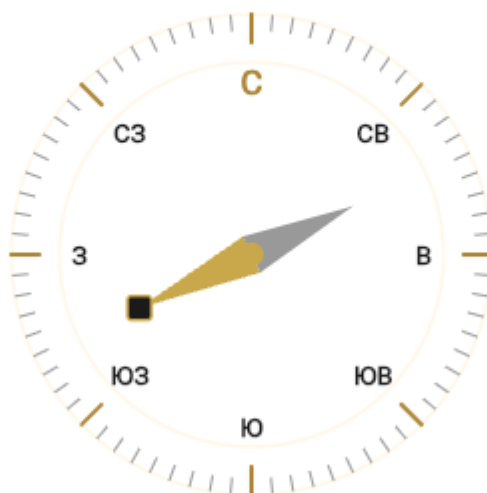
Для модели PrayerSchedule проверялись координаты, название местоположения, пять обязательных намазов, восход, следующий намаз и время до него. Значения использовались как контрольные точки при проверке экрана и связанных уведомлений.

Взаимодействие компонентов показано на рисунке 7.

Намаз

Ваше местоположение · Бишкек

Направление киблы · Бишкек



245° ЮЗ

Датчик компаса недоступен или ожидает калибровки



Расписание на сегодня



☒ Фаджр

الفجر 04:53

☒ Восход

الشروق 06:32

☐ Зухр

الظهر 13:01

☐ Аср

العصر 17:32



Главная



Намаз



Уроки



Коран



Профиль

Рисунок 7 — Взаимодействие компонентов религиозного модуля

3.2 Проверка получения географических координат

Проверка геолокации выполнялась при включённой и выключенной службе, разрешённом и запрещённом доступе, окончательном запрете и превышении пятисекундного тайм-аута. В каждом случае контролировались источник координат и отсутствие аварийного завершения.

В положительном сценарии `Geolocator.getCurrentPosition` возвращал текущую позицию, после чего на экране отображались соответствующие координаты и рассчитанное расписание. Запрос разрешения выполнялся только при неопределённом состоянии доступа.

При отключённой службе, отказе, тайм-ауте или исключении применялись резервные координаты Бишкека 42.8746 и 74.5698. Экран сохранял работоспособность и явно показывал используемое местоположение. Результаты приведены в таблице 8 и на рисунке 8.

Таблица 8 — Сценарии получения местоположения

Условие	Действие сервиса	Используемые координаты
служба включена, разрешение выдано	запрос позиции с высокой точностью	фактические координаты устройства
разрешение ещё не запрашивалось	показ системного запроса разрешения	координаты устройства при согласии
служба выключена или доступ запрещён	перехват исключения и резервный сценарий	Бишкек: 42,8746; 74,5698
ответ не получен за пять секунд	завершение ожидания по тайм-ауту	Бишкек: 42,8746; 74,5698

Отображение местоположения показано на рисунке 8.

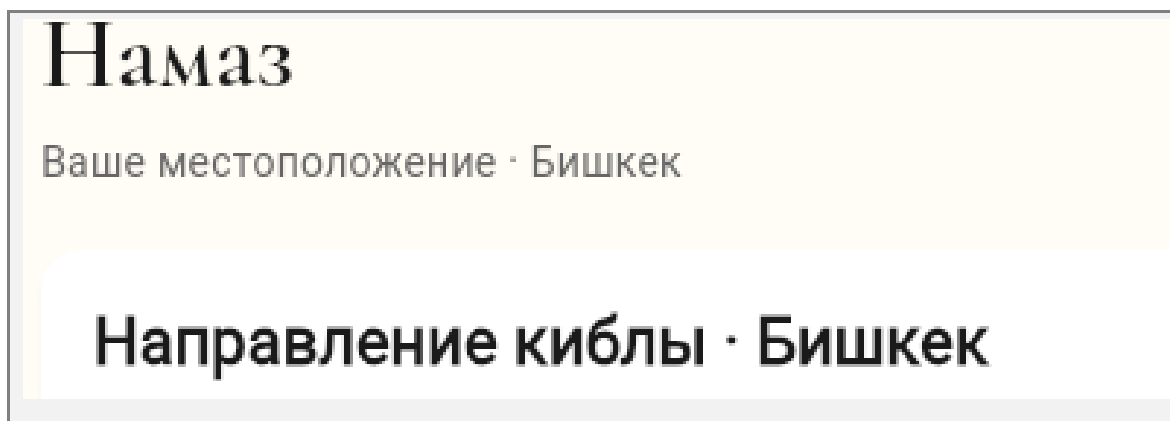


Рисунок 8 — Отображение местоположения в расписании намазов

3.3 Проверка расписания и следующего намаза

Для расчёта проверялись координаты, текущая дата, метод Muslim World League и ханафитский способ определения Асра. Времена пяти намазов сравнивались с объектом PrayerTimes, а восход учитывался как вспомогательное событие.

На экране проверялись состояния ожидания, результата и ошибки, формат времени, выделение следующего намаза и обновление обратного отсчёта. Проверяемые состояния перечислены в таблице 9.

Таблица 9 — Проверяемые состояния расписания намазов

Значение PrayerStatus	Назначение	Отображение
done	время уже наступило	завершённый пункт
next	ближайший будущий обязательный намаз	акцентная строка
upcoming	последующий обязательный намаз	обычная строка
faint	вспомогательное время Восход	приглушённая строка

Граничный сценарий выполнялся после Иша. В текущем дне будущего обязательного намаза уже нет, поэтому сервис должен выбрать Фаджр следующего дня. Повторная проверка подтвердила корректную дату, название и положительный обратный отсчёт.

Расписание и следующий намаз показаны на рисунке 9.

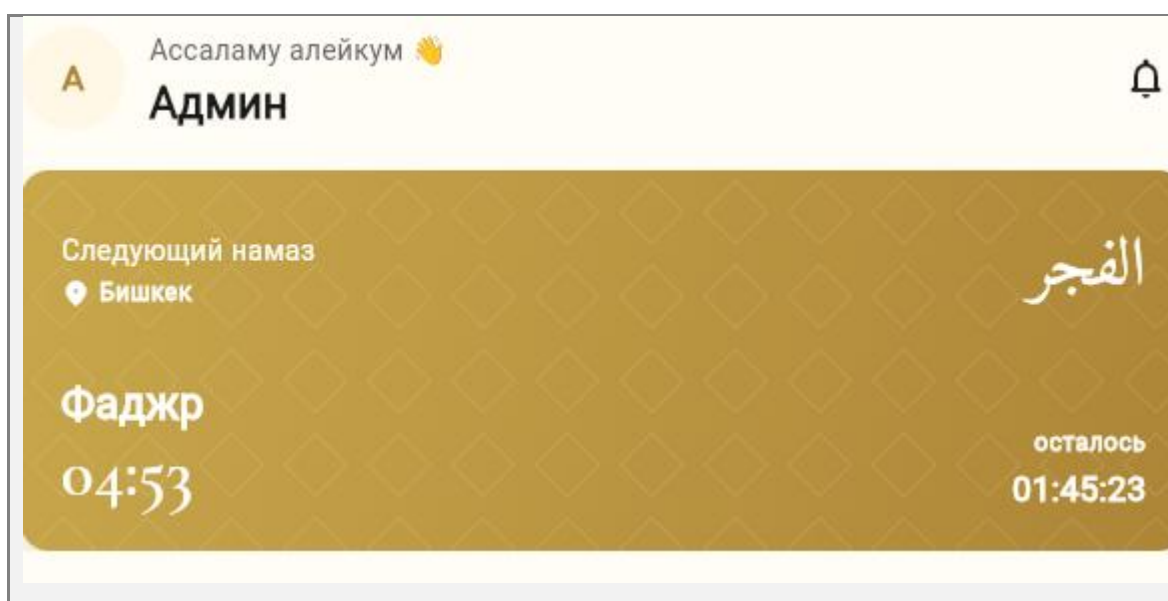


Рисунок 9 — Расписание, следующий намаз и обратный отсчёт

3.4 Проверка направления Кыблы

Для известных координат проверялся абсолютный азимут на Каабу, вычисленный классом Qibla. Затем результат сопоставлялся с текущим heading устройства, а относительный угол приводился к диапазону от 0 до 360 градусов.

При изменении положения устройства поток Flutter Compass обновлял направление стрелки. Проверялись повороты через север и значения около границ диапазона, где особенно важна правильная нормализация угла.

Негативный сценарий выполнялся без данных датчика. Виджет не завершал работу с ошибкой и сохранял отображение абсолютного азимута. Результат проверки направления Кыблы зафиксирован на рисунке 10.



Рисунок 10 — Определение направления Кыблы по данным компаса

3.5 Проверка локальных напоминаний

На Android проверялись инициализация NotificationService, создание канала, формирование времени срабатывания и передача названия намаза. На

Web проверялось, что неподдерживаемый платформенный метод не вызывается.

Перед созданием нового расписания проверялась отмена ранее запланированных уведомлений. Для будущих намазов формировались отдельные идентификаторы, а прошедшие события не планировались повторно.

Исходный Web-сценарий воспроизводил исключение `Unsupported operation` при вызове `zonedSchedule`. После добавления проверки `kIsWeb` ошибка перестала возникать. На Android дополнительно подтверждался резервный переход от точного к обычному расписанию уведомления.

Таблица 10 — Результаты платформенной проверки уведомлений

Платформа	Поведение	Причина
Android	инициализация канала, запрос разрешений и планирование будущих намазов	плагин поддерживает системное <code>zonedSchedule</code>
Web	методы инициализации и планирования завершаются без вызова плагина	<code>zonedSchedule</code> не поддерживается браузерной реализацией

Локальное Android-уведомление показано на рисунке 11.

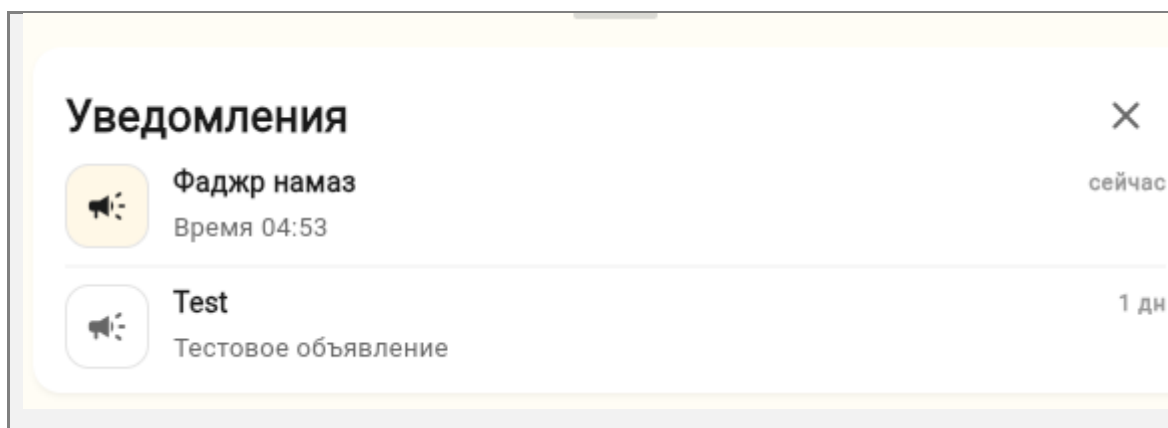


Рисунок 11 — Локальное напоминание о наступлении намаза на Android

3.6 Платформенное тестирование на Android и Web

Платформенная проверка `PrayerScreen` охватывала состояния загрузки, результата и ошибки. На обеих платформах интерфейс оставался доступным, а различия системных API обрабатывались до выполнения неподдерживаемого вызова.

Отказ геолокации приводил к резервным координатам, отсутствие компаса — к безопасному состоянию виджета, а Web-версия пропускала локальное планирование. Эти сценарии подтверждали устойчивость без подмены основной логики расчёта.

Повторный запуск Web-версии после исправления не содержал исключения `zonedSchedule`. Рабочий экран религиозного модуля представлен на рисунке 12.

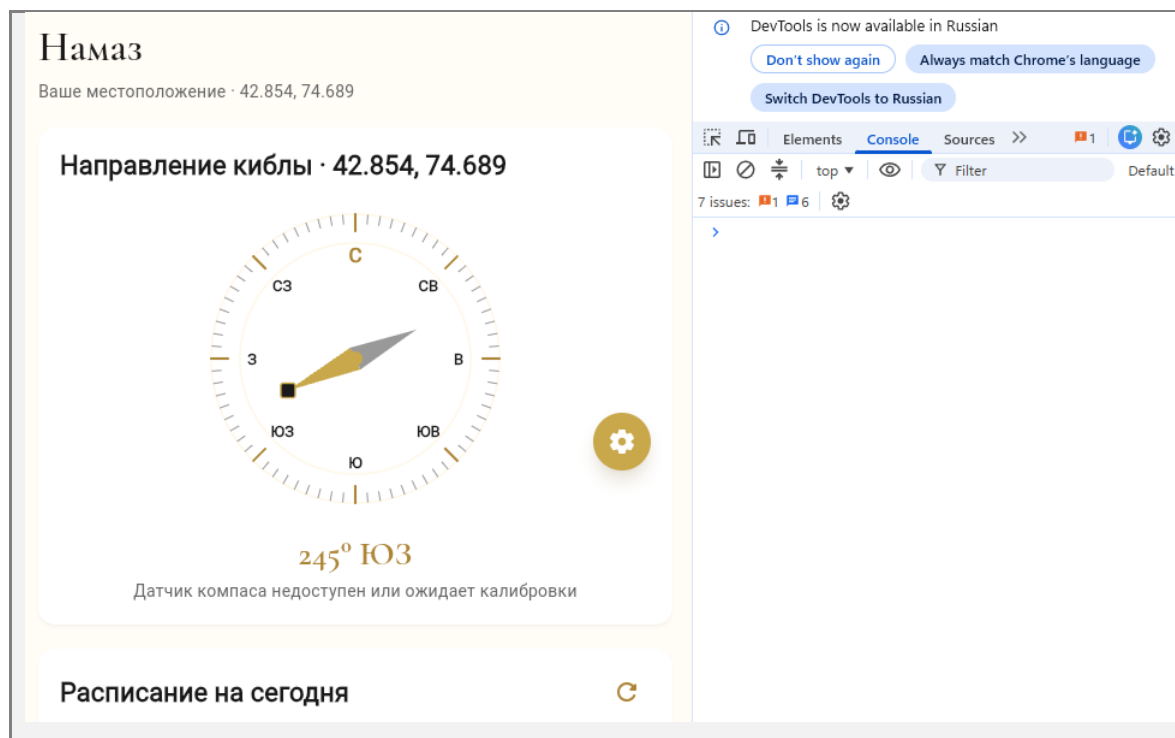


Рисунок 12 — Результат повторной проверки Web-версии

4 АВТОМАТИЗИРОВАННОЕ И РЕГРЕССИОННОЕ ТЕСТИРОВАНИЕ

4.1 Методика и уровни тестирования

Методика включала статический анализ, автоматизированные тесты, ручные функциональные сценарии, негативные проверки и регрессию после исправлений. Для внешних сервисов и датчиков использовалась фактическая среда, поскольку widget-тест не измеряет сеть, GPS, компас и доставку уведомлений.

Каждый сценарий фиксировал начальные условия, последовательность действий, ожидаемый результат и фактическое поведение. Исправленный дефект считался закрытым только после повторного воспроизведения исходных шагов и успешной базовой регрессии.

4.2 Среда и результаты автоматизированных проверок

Проверка выполнена 7 сентября 2026 года на Windows с Flutter 3.44.9 stable, Dart 3.12.2 и Node.js. Команды запускались из корневого каталога проекта. Полученные результаты приведены в таблице 11.

Таблица 11 — Результаты автоматизированных проверок проекта

Команда	Назначение	Результат 07.09.2026
flutter analyze	статический анализ Dart-кода и правил flutter_lints	замечаний не обнаружено; 135 секунд
flutter test	запуск существующего набора модульных и widget-тестов	2 теста из 2 пройдены; около 2 секунд
npm run check	проверка синтаксиса functions/index.js командой node --check	завершено успешно, синтаксических ошибок нет

Статический анализ завершён сообщением No issues found. Это подтверждает соответствие текущего Dart-кода подключённым правилам flutter_lints. Результат команды показан на рисунке 13.

```
HP@unknown MINGW64 ~/OneDrive/Desktop/irfan_academy
$ flutter analyze
Analyzing irfan_academy...
No issues found! (ran in 60.8s)
```

Рисунок 13 — Результат статического анализа Flutter

4.3 Автоматизированное тестовое покрытие

Автоматизированный набор содержит два теста: распознавание административной роли и отображение стартового экрана. Оба теста завершились успешно. Их назначение и результат приведены в таблице 12.

Таблица 12 — Существующие автоматизированные тесты Flutter

Код	Проверяемое поведение	Фактический результат
АТ-01	роли admin и «Администратор» распознаются как административные, роль «Ученик» — нет	пройден
АТ-02	SplashScreen содержит название, кнопку начала обучения и подпись местоположения	пройден

Эти тесты обеспечивают базовую регрессионную проверку приложения. Сетевые, геолокационные и платформенные сценарии проверялись отдельно вручную, поскольку зависят от внешней среды и аппаратных возможностей.

```
HP@unknown MINGW64 ~/OneDrive/Desktop/irfan_academy
$ flutter test
00:08 +2: All tests passed!
```

Рисунок 14 — Успешное выполнение автоматизированных тестов Flutter

4.4 Регрессионная проверка видеомодуля

После устранения дефектов видеомодуль повторно проверялся по сценарию от выбора файла до воспроизведения. Регрессия включала оба режима загрузки, индикатор прогресса, метаданные, Firestore и элементы управления плеером. Итоги приведены в таблице 13.

Таблица 13 — Проверенные сценарии видеомодуля

Код	Действия	Полученный результат
VT-01	открыть системный выбор видео и отменить операцию	форма остаётся открытой, загрузка и запись не создаются
VT-02	выбрать корректный MP4 размером меньше 95 МБ	запускается потоковая загрузка, процент достигает 100
VT-03	загрузить видео размером больше 95 МБ	файл передаётся частями по 8 МБ без загрузки целиком в память
VT-04	временно прервать соединение при отправке части	выполняются повторные попытки с увеличением задержки
VT-05	отменить частичную загрузку	цикл прекращается на ближайшей проверке, урок не публикуется
VT-06	завершить загрузку и сохранить номер, название и описание	Firestore получает метаданные, но не бинарное содержимое файла
VT-07	открыть сохранённый урок и использовать паузу, перемотку и скорость	видео воспроизводится, элементы управления изменяют состояние контроллера
VT-08	передать недоступный сетевой адрес и нажать «Повторить»	показывается сообщение об ошибке без аварийного закрытия интерфейса

4.5 Регрессионная проверка религиозного модуля

Регрессионная проверка религиозного модуля охватывала доступную и отключённую геолокацию, резервные координаты, переход после Иша, отсутствие компаса и Web-запуск без zonedSchedule. Итоги приведены в таблице 14.

Таблица 14 — Проверенные сценарии религиозного модуля

Код	Условие проверки	Полученный результат
РТ-01	геолокация включена и разрешение выдано	расписание строится по фактическим координатам
РТ-02	геолокация выключена либо доступ запрещён	за пять секунд выбираются резервные координаты Бишкека
РТ-03	открыть расписание до наступления ближайшего намаза	один обязательный намаз получает состояние next
РТ-04	определить следующий намаз после Иша	выбирается Фаджр следующего дня, отсчёт остаётся положительным

Код	Условие проверки	Полученный результат
PT-05	поворачивать устройство с доступным компасом	указатель Кыблы изменяет относительное направление
PT-06	открыть экран на устройстве без данных компаса	отображаются абсолютный азимут и сообщение о датчике
PT-07	разрешить уведомления на Android	будущие обязательные намазы планируются локально
PT-08	открыть модуль в Web-версии	zonedSchedule не вызывается, экран работает без исключения

4.6 Регистрация и повторная проверка дефектов

Обнаруженные дефекты фиксировались с условиями воспроизведения, фактическим и ожидаемым результатом. После исправления исходные шаги выполнялись повторно. Закрытые в ходе работы дефекты перечислены в таблице 15.

Таблица 15 — Дефекты, устранённые в индивидуальных модулях

Дефект	Причина	Внесённое изменение
исключение zonedSchedule в браузере	мобильный API вызывался в Web-реализации	добавлена ранняя проверка kIsWeb
видеоплеер имитировал воспроизведение	использовался локальный таймер вместо медиаконтроллера	подключён VideoPlayerController.networkUrl
URL и длительность вводились вручную	технические данные не извлекались из ответа хранилища	поля заполняются по ответу Cloudinary
риск большого потребления памяти	крупный файл мог обрабатываться как единый массив	добавлены потоковая передача и части по 8 МБ
после Иша выбирался прошедший Фаджр	поиск выполнялся только в расписании текущего дня	добавлен расчёт Фаджра следующего дня
экран зависел от GPS и компаса	не учитывались отказ в разрешении и отсутствие датчика	добавлены резервные координаты и безопасное состояние компаса

После повторных проверок выполнены flutter analyze и flutter test. Статический анализ завершился без замечаний, а два существующих теста прошли успешно. Дополнительно node --check подтвердил корректный синтаксис файла Firebase Functions.

5 ИТОГОВЫЕ РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

5.1 Результаты тестирования видеомодуля

Проверка видеомодуля подтвердила выбор файла, автоматический запуск, отображение прогресса, потоковую и блочную передачу, повтор временных ошибок и получение метаданных Cloudinary. Документ Firestore содержал только описание урока и технические сведения, а видео оставалось во внешнем хранилище.

Сетевой плеер открывал опубликованный урок и выполнял запуск, паузу, перемотку, повтор и изменение скорости. Негативные состояния отображались управляемо, а ресурсы контроллера освобождались при смене или закрытии урока.

5.2 Результаты тестирования религиозного модуля

Проверка PrayerService подтвердила получение координат, резервный переход к Бишкеку и расчёт расписания по методу Muslim World League с ханафитским Асром. После Иша следующим событием корректно определялся Фаджр следующего дня.

Направление Кыблы обновлялось по данным компаса, а отсутствие датчика не приводило к сбою. Android-уведомления планировались локально, Web-версия пропускала неподдерживаемый вызов `zonedSchedule` и продолжала работу.

5.3 Сводный результат тестирования

Выполненные функциональные, негативные, платформенные и автоматизированные проверки подтвердили работоспособность тестируемых модулей в заявленных сценариях. Статический анализ, два теста Flutter и синтаксическая проверка Functions завершились успешно. Общий итог приведён в таблице 16.

Таблица 16 — Итоги тестирования мультимедийных и религиозных модулей

Направление	Выполненная работа	Полученный результат
-------------	--------------------	----------------------

Направление	Выполненная работа	Полученный результат
Загрузка видео	выбор файла, потоковая и частичная передача, прогресс и повторы	видео загружается в Cloudinary, параметры определяются автоматически
Воспроизведение	подключение VideoPlayerController.networkUrl и элементов управления	вместо таймера используется настоящий сетевой видеоплеер
Расписание намазов	геолокация, резервные координаты и расчёт библиотекойadhan	отображаются времена намазов и ближайший намаз
Кыбла	расчёт азимута и обработка показаний компаса	указатель реагирует на направление устройства
Напоминания	локальное планирование с отдельной обработкой Web	Android получает напоминания, браузер не вызывает zonedSchedule
Контроль качества	статический анализ, Flutter-тесты и проверка Functions	анализ без замечаний, 2 теста пройдены, синтаксис Functions корректен

ЗАКЛЮЧЕНИЕ

В ходе производственной практики выполнено тестирование мультимедийных и религиозных модулей Irfan Academy. Проверены загрузка и воспроизведение видео, геолокация, расписание намазов, направление Кыблы и локальные уведомления.

Тестирование охватило положительные, негативные и граничные состояния: большие видеофайлы, временные сетевые ошибки, отказ разрешений, отсутствие компаса, переход после Иша и различия Android и Web. Выявленные ошибки zonedSchedule и имитации воспроизведения были устранены и повторно проверены.

Контроль качества завершён статическим анализом, автоматизированными тестами Flutter и проверкой синтаксиса Firebase Functions. Все выполненные проверки дали успешный результат, а существующая архитектура Firebase Authentication и Firestore сохранила работоспособность.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления. — URL:
<https://protect.gost.ru/gost/details/7d280e43-7036-4a69-8e6e-d15867028343>
2. Кыргызский государственный технический университет им. И. Раззакова. Положение о порядке организации практик студентов. — URL:
https://kstu.kg/fileadmin/user_upload/13._polozhenie_o_porjadke_organizacii_praktik_studentov_kgtu_im._i._razzakova__2020_g._.pdf
3. Flutter. Официальная документация. — URL: <https://docs.flutter.dev/>
4. Dart. Официальная документация языка и библиотек. — URL:
<https://dart.dev/guides>
5. Firebase Authentication for Flutter. Официальная документация. — URL:
<https://firebase.google.com/docs/auth/flutter/start>
6. Cloud Firestore for Flutter. Официальная документация. — URL:
<https://firebase.google.com/docs/firestore/quickstart>
7. Firebase Callable Functions. Официальная документация. — URL:
<https://firebase.google.com/docs/functions/callable>
8. Cloudinary Upload API Reference. — URL:
https://cloudinary.com/documentation/image_upload_api_reference
9. Cloudinary Chunked Asset Upload. — URL:
https://cloudinary.com/documentation/upload_images#chunked_asset_upload
10. Пакет video_player для Flutter. — URL:
https://pub.dev/packages/video_player
11. Пакет geolocator для Flutter. — URL: <https://pub.dev/packages/geolocator>
12. Пакет adhan для Dart. — URL: <https://pub.dev/packages/adhan>
13. Пакет flutter_compass для Flutter. — URL:
https://pub.dev/packages/flutter_compass

14. Пакет flutter_local_notifications для Flutter. — URL:
https://pub.dev/packages/flutter_local_notifications

ПРИЛОЖЕНИЕ А

А.1 Файлы, исследованные при тестировании

Таблица А.1 — Файлы, исследованные при выполнении тестирования

Файл	Выполненная работа	Назначение
cloudinary_video_service.dart	потокковая и частичная загрузка, прогресс, повторы и метаданные	передача видео в Cloudinary
video_player_modal.dart	сетевой контроллер и элементы управления	воспроизведение видеоуроков
prayer_service.dart	геолокация, намазы, следующий день и Кыбла	формирование PrayerSchedule
notification_service.dart	часовой пояс, разрешения и планирование	локальные напоминания
qibla_compass.dart	обработка heading и отрисовка указателя	визуальное направление Кыблы
admin_screen.dart	выбор файла, диалог прогресса и форма урока	административный сценарий загрузки
prayer_screen.dart	загрузка расписания и обновление статусов	экран религиозных функций
pubspec.yaml	подключение пакетов видео, геолокации и уведомлений	зависимости индивидуальных модулей

ПРИЛОЖЕНИЕ Б. ЧЕК-ЛИСТ ДЕМОНСТРАЦИИ

1. Запустить приложение и выполнить вход администратором.
2. Открыть административную панель и раздел видеоуроков.
3. Выбрать MP4-файл и показать индикатор загрузки.
4. Заполнить метаданные урока и сохранить запись.
5. Открыть Firestore и показать, что хранится только URL и метаданные.
6. Войти учеником и воспроизвести опубликованный урок.
7. Показать расписание намазов и обратный отсчёт.
8. Показать направление Кыблы.
9. Показать успешный результат flutter analyze и flutter test.
10. Предъявить release APK и GitHub-репозиторий.